ZHENGZHOU UNIVERSITY

School of Computer and Artificial Intelligence

Design and Analysis of Algorithms

Lab Practice 03 | Bubble Sort and Selection Sort

Instructor: Dr. Hussain | **Language:** C++ or Python | **Lab type:** Hands-on

1. Learning objectives

Learn how sorting algorithms arrange values in ascending order. Write pseudocode, translate it into a working program, trace the execution, and test the results. Big O notation is not required.

• Explain Bubble Sort and Selection Sort in simple terms.

• Write original pseudocode for each algorithm before coding.

• Implement both algorithms in C++ or Python.

• Print the array after each pass, count comparisons and actual swaps, and test different inputs.

Lab rule: Write pseudocode and program code manually. Do not copy and paste code. You must be ready to explain your solution.

2. Task 1: Bubble Sort

A. Understand: Bubble Sort compares adjacent values and swaps them when the left value is greater. After each complete pass, the largest remaining value moves to its final position.

B. Write pseudocode yourself: Use FUNCTION, FOR, IF, SWAP, and END statements. Include an outer pass loop, an inner loop for adjacent comparisons, and a swap condition. Do not start programming until your pseudocode is complete.

C. Write the program: Store 10 integers, display the original array, sort in ascending order, print the array after every pass, and display the final sorted array.

Input: 64, 34, 25, 12, 22, 11, 90, 45, 8, 30

Expected final output: 8, 11, 12, 22, 25, 30, 34, 45, 64, 90

3. Task 2: Selection Sort

A. Understand: Selection Sort finds the smallest value in the unsorted portion and exchanges it with the first position of that portion.

B. Write pseudocode yourself: Use FUNCTION, FOR, IF, SWAP, and END statements. Track the minimum index, search the unsorted portion, and exchange elements when needed.

C. Write the program: Use the same 10 integers as Task 1. Display the original array, the array after every pass, and the final sorted array.

Expected final output: 8, 11, 12, 22, 25, 30, 34, 45, 64, 90

4. Task 3: Manual tracing

Given the array **[5, 3, 8, 4, 2]**, trace both algorithms manually. Record the complete array after each pass and identify the elements swapped. Check your work against program output.

Pass	Bubble Sort array	Selection Sort array
Initial	5, 3, 8, 4, 2	5, 3, 8, 4, 2

Pass	Bubble Sort array	Selection Sort array
1		
2		
3		
4		

5. Task 4: Count comparisons and swaps

Modify both programs to count comparisons between array values and actual exchanges of elements at different positions. Do not count a self-swap as an actual swap. Use the same input for both algorithms.

Algorithm	Comparisons	Actual swaps
Bubble Sort		
Selection Sort		

Explain which algorithm performed fewer swaps for your input.

6. Task 5: Modify and test

Run both programs on all three inputs. Record the sorted output, number of comparisons, number of swaps, and output screenshots.

Test	Input
Already sorted	1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Reverse sorted	10, 9, 8, 7, 6, 5, 4, 3, 2, 1
Duplicate values	5, 3, 5, 2, 8, 2, 9, 1, 4, 3

7. Questions students should answer

1. What is the difference between Bubble Sort and Selection Sort?
2. Why does Bubble Sort compare adjacent elements?
3. How does Selection Sort find the smallest element?
4. Which algorithm performed fewer swaps in your experiments?
5. What happens when the input array is already sorted?
6. What happens when the input array is in reverse order?
7. Can both algorithms sort in descending order? What needs to change?
8. Why is manual tracing useful when learning sorting algorithms?

Answer using your own words and observations. Big O notation is not required.

8. Final task and submission

Complete all five practical tasks in C++ or Python. Create one folder named with your **Student ID and Name**. Include:

• BubbleSort.cpp or BubbleSort.py

• SelectionSort.cpp or SelectionSort.py

• Lab03_Report.pdf containing both handwritten/typed original pseudocodes, manual tracing, output screenshots, comparison tables, and answers to all eight questions.

Submit the folder to your class monitor before the end of class. All pseudocode and program code must be written manually; copying and pasting is not allowed.